

EE 446 TinyML Compression Cookbook

A Practical TensorFlow Template for FFNNs, Training, Quantization, Pruning, and Knowledge Distillation

This document summarizes the key coding patterns used across the EE 446 lab notebooks on quantization, pruning, and knowledge distillation. The goal is to give students a compact implementation guide they can reuse in their own projects with minimal changes.

Important note. The uploaded lab notebooks are based on the UCI HAR dataset, which is a *tabular numerical-feature* dataset. Therefore, the exact lab implementations use dense neural networks rather than CNNs. A simple CNN template is still included here because many final projects use image-like or time-series tensor inputs, and students often need a ready-to-adapt starting point.

Contents

1	Defining TensorFlow Models: FFNN and Simple CNN	2
1.1	Feedforward Neural Network (FFNN) for tabular or engineered features	2
1.2	Simple CNN for image-like or spatial inputs	2
2	Training, Validation, Testing, and Evaluation	3
2.1	Compiling and training the model	3
2.2	Testing and reporting performance	4
2.3	Evaluating TensorFlow Lite models	4
3	Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT)	5
3.1	Representative dataset for full integer quantization	5
3.2	Reusable PTQ conversion helpers	6
3.3	Running PTQ on one trained model	6
3.4	Quantization-Aware Training (QAT)	7
4	Pruning: Scheduling Sparsity, Stripping the Mask, and Combining with Quantization	7
4.1	Setting up magnitude-based pruning	7
4.2	Training a pruned model	8
4.3	Why stripping the pruning mask matters	9
4.4	Combining pruning with float16 quantization	9
4.5	Pruning a distilled student	10
5	Knowledge Distillation (Output-Based)	10

5.1	Teacher and student models	10
5.2	Baseline student training before distillation	11
5.3	Custom distillation model wrapper	11
5.4	Compiling and training the distiller	13
5.5	Final compression after distillation	13

1. Defining TensorFlow Models: FFNN and Simple CNN

1.1 Feedforward Neural Network (FFNN) for tabular or engineered features

This is the main model style used in the lab notebooks. It is appropriate when each example is represented by a fixed-length feature vector.

```
def build_ffnn_model(input_dim, num_classes):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(256, activation="relu"),
        layers.Dense(128, activation="relu"),
        layers.Dense(64, activation="relu"),
        layers.Dense(num_classes, activation="softmax")
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model

model = build_ffnn_model(input_dim=num_features, num_classes=num_classes)
model.summary()
```

Why this pattern matters. This is a strong default for numerical datasets. The lab notebooks use `sparse_categorical_crossentropy`, which is appropriate when labels are integer-encoded such as 0,1,2,...

When to reuse it. Use this pattern when your data is already flattened into features, such as sensor statistics, TF-IDF vectors, tabular biomedical features, or classical ML feature engineering outputs.

1.2 Simple CNN for image-like or spatial inputs

This template is not directly used in the HAR notebooks, but it is a practical companion pattern for course projects that use image or 2D tensor inputs.

```
def build_simple_cnn_model(input_shape, num_classes):
    model = keras.Sequential([
        layers.Input(shape=input_shape),
        layers.Conv2D(16, kernel_size=3, padding="same", activation="relu"),
        layers.MaxPooling2D(pool_size=2),
        layers.Conv2D(32, kernel_size=3, padding="same", activation="relu"),
```

```

        layers.MaxPooling2D(pool_size=2),
        layers.Conv2D(64, kernel_size=3, padding="same", activation="relu"),
        layers.Flatten(),
        layers.Dense(64, activation="relu"),
        layers.Dense(num_classes, activation="softmax")
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model

cnn_model = build_simple_cnn_model(input_shape=(32, 32, 3), num_classes=10)
cnn_model.summary()

```

Why this pattern matters. CNNs are usually better than dense-only models when local spatial structure matters, such as pixels, spectrograms, or small 2D sensor maps.

Practical reminder. If your input is 1D sequential data rather than 2D images, consider using Conv1D instead of Conv2D, but keep the same overall design logic.

2. Training, Validation, Testing, and Evaluation

2.1 Compiling and training the model

The lab notebooks use Adam, early stopping, and a validation split. This is a clean pattern for fast iteration.

```

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=5,
        restore_best_weights=True
    )
]

history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=40,
    batch_size=64,
    callbacks=callbacks,
    verbose=1
)

```

```
)
```

Key implementation ideas.

- `validation_split=0.2` reserves part of the training set for model selection.
- `EarlyStopping` prevents unnecessary overtraining and restores the best weights.
- `batch_size=64` is a practical default used repeatedly in the labs.

2.2 Testing and reporting performance

The notebooks evaluate classification performance using both a classification report and a confusion matrix.

```
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

probs = model.predict(X_test, verbose=0)
preds = np.argmax(probs, axis=1)
test_acc = accuracy_score(y_test, preds)

print(f"Test Accuracy: {test_acc:.4f}\n")
print(classification_report(y_test, preds, target_names=class_names, digits=4))

disp = ConfusionMatrixDisplay(
    confusion_matrix=confusion_matrix(y_test, preds),
    display_labels=class_names
)
fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, xticks_rotation=45, cmap="Blues", colorbar=False)
plt.title("Confusion Matrix")
plt.show()
```

Why both are useful.

- **Accuracy** gives an overall summary.
- **Classification report** shows precision, recall, and F1 per class.
- **Confusion matrix** shows which classes are being mixed up.

2.3 Evaluating TensorFlow Lite models

Once a model is converted to TFLite, you cannot evaluate it with `model.predict(...)` directly. The labs therefore use an interpreter-based evaluation loop.

```
def evaluate_tflite_model(tflite_model, X, y_true):
    interpreter = tf.lite.Interpreter(model_content=tflite_model)
    interpreter.allocate_tensors()
```

```

input_details = interpreter.get_input_details()[0]
output_details = interpreter.get_output_details()[0]

input_scale, input_zero_point = input_details["quantization"]
output_scale, output_zero_point = output_details["quantization"]

y_pred = []

for i in range(len(X)):
    x = X[i:i+1].astype(np.float32)

    if input_details["dtype"] == np.int8:
        x = np.round(x / input_scale + input_zero_point).astype(np.int8)
    elif input_details["dtype"] == np.uint8:
        x = np.round(x / input_scale + input_zero_point).astype(np.uint8)
    else:
        x = x.astype(input_details["dtype"])

    interpreter.set_tensor(input_details["index"], x)
    interpreter.invoke()

    output = interpreter.get_tensor(output_details["index"])

    if output_details["dtype"] in (np.int8, np.uint8):
        output = (output.astype(np.float32) - output_zero_point) * output_scale

    y_pred.append(np.argmax(output, axis=1)[0])

y_pred = np.array(y_pred)
acc = accuracy_score(y_true, y_pred)
return acc, y_pred

```

Why this helper is important. For quantized models, the input and output tensors may be stored as integer values. The interpreter loop must therefore handle quantization and dequantization correctly before predictions are compared with the ground-truth labels.

3. Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT)

3.1 Representative dataset for full integer quantization

The int8 PTQ and int8 QAT conversion paths use representative data so that TensorFlow Lite can estimate activation ranges.

```

def representative_dataset_gen():
    for i in range(300):
        yield [X_train[i:i+1].astype(np.float32)]

```

Practical note. Representative samples should reflect the true range and distribution of the training data. Poor representative data often hurts int8 accuracy.

3.2 Reusable PTQ conversion helpers

The quantization lab uses separate helper functions for each conversion mode.

```
def convert_to_tflite_fp32(model):
    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    return converter.convert()

def convert_to_tflite_dynamic_range(model):
    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    return converter.convert()

def convert_to_tflite_float16(model):
    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.target_spec.supported_types = [tf.float16]
    return converter.convert()

def convert_to_tflite_int8(model):
    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.representative_dataset = representative_dataset_gen
    converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    converter.inference_input_type = tf.int8
    converter.inference_output_type = tf.int8
    return converter.convert()
```

How to interpret the four formats.

- **FP32 TFLite:** mostly a reference conversion of the original floating-point model.
- **Dynamic range PTQ:** weights are quantized, but activations are quantized dynamically at runtime.
- **Float16 PTQ:** a useful size reduction option that often preserves accuracy well.
- **Full int8 PTQ:** strongest compression and deployment-friendliness, but also the most sensitive to calibration quality.

3.3 Running PTQ on one trained model

The lab uses the following pattern to create and evaluate all post-training variants.

```
fp32_tflite_model = convert_to_tflite_fp32(model)
drq_tflite_model = convert_to_tflite_dynamic_range(model)
float16_tflite_model = convert_to_tflite_float16(model)
int8_tflite_model = convert_to_tflite_int8(model)

fp32_size_kb = save_binary_model(fp32_tflite_model, "baseline_fp32.tflite")
drq_size_kb = save_binary_model(drq_tflite_model, "baseline_dynamic_range.tflite")
float16_size_kb = save_binary_model(float16_tflite_model, "baseline_float16.tflite")
int8_size_kb = save_binary_model(int8_tflite_model, "baseline_int8.tflite")

fp32_acc, fp32_preds = evaluate_tflite_model(fp32_tflite_model, X_test, y_test)
```

```
drq_acc, drq_preds = evaluate_tflite_model(drq_tflite_model, X_test, y_test)
float16_acc, float16_preds = evaluate_tflite_model(float16_tflite_model, X_test, y_test)
int8_acc, int8_preds = evaluate_tflite_model(int8_tflite_model, X_test, y_test)
```

3.4 Quantization-Aware Training (QAT)

QAT inserts fake quantization behavior during training so the model can adapt before final int8 conversion.

```
qat_model = tfmot.quantization.keras.quantize_model(model)

qat_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

qat_history = qat_model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=8,
    batch_size=64,
    verbose=1
)
```

After fine-tuning, the QAT model is converted using the same full-int8 conversion logic:

```
qat_int8_tflite_model = convert_to_tflite_int8(qat_model)
qat_int8_size_kb = save_binary_model(qat_int8_tflite_model, "qat_int8.tflite")
qat_int8_acc, qat_int8_preds = evaluate_tflite_model(
    qat_int8_tflite_model,
    X_test,
    y_test
)
```

Important implementation idea. The lab uses PTQ for FP32, dynamic range, float16, and int8. QAT is then used specifically to improve the final int8 deployment model.

4. Pruning: Scheduling Sparsity, Stripping the Mask, and Combining with Quantization

4.1 Setting up magnitude-based pruning

The pruning lab uses a polynomial decay schedule. This gradually increases sparsity during training.

```
from tensorflow_model_optimization.sparsity.keras import (
    prune_low_magnitude,
    PolynomialDecay,
    UpdatePruningStep,
    strip_pruning
)
```

```

pruning_epochs = 12
batch_size = 64
steps_per_epoch = math.ceil((0.8 * len(X_train)) / batch_size)

pruning_params = {
    "pruning_schedule": PolynomialDecay(
        initial_sparsity=0.20,
        final_sparsity=0.85,
        begin_step=0,
        end_step=steps_per_epoch * pruning_epochs
    )
}

```

How to think about these parameters.

- `initial_sparsity`: how sparse the model starts.
- `final_sparsity`: how sparse the model should be by the end of pruning.
- `begin_step`: when pruning starts.
- `end_step`: when the target sparsity is reached.

Practical advice. A very aggressive `final_sparsity` may hurt accuracy. A reasonable workflow is to test a few values such as 0.50, 0.70, 0.85, and compare the accuracy–size tradeoff.

4.2 Training a pruned model

```

pruned_model = prune_low_magnitude(
    build_ffnn_model(num_features, num_classes),
    **pruning_params
)

pruned_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

pruning_callbacks = [
    UpdatePruningStep(),
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=3,
        restore_best_weights=True
    )
]

pruning_history = pruned_model.fit(
    X_train,
    y_train,
    validation_split=0.2,

```



```

    epochs=pruning_epochs,
    batch_size=batch_size,
    callbacks=pruning_callbacks,
    verbose=1
)

```

Important reminder. `UpdatePruningStep()` is essential. Without it, the pruning schedule does not progress correctly during training.

4.3 Why stripping the pruning mask matters

A pruned model still contains wrapper-related information until the pruning wrappers are removed. This is why a pruned model with the mask still attached often does *not* give the final compressed file size students expect.

```

pruned_with_mask_tflite = convert_to_tflite_fp32(pruned_model)
pruned_with_mask_size_kb = save_binary_model(
    pruned_with_mask_tflite,
    "pruned_with_mask_fp32.tflite"
)

stripped_model = strip_pruning(pruned_model)

converter = tf.lite.TFLiteConverter.from_keras_model(stripped_model)
converter.optimizations = [tf.lite.Optimize.EXPERIMENTAL_SPARSITY]
stripped_sparse_tflite = converter.convert()

stripped_sparse_size_kb = save_binary_model(
    stripped_sparse_tflite,
    "stripped_sparse_fp32.tflite"
)

```

Core lesson. Pruning is not enough by itself. To get an actual deployment-friendly sparse model size reduction:

1. train with pruning wrappers,
2. strip the pruning wrappers,
3. convert with sparse optimization enabled.

4.4 Combining pruning with float16 quantization

The pruning notebook then combines pruning with float16 conversion.

```

converter = tf.lite.TFLiteConverter.from_keras_model(stripped_model)
converter.optimizations = [
    tf.lite.Optimize.DEFAULT,
    tf.lite.Optimize.EXPERIMENTAL_SPARSITY
]
converter.target_spec.supported_types = [tf.float16]

stripped_sparse_float16_tflite = converter.convert()

```

```

stripped_sparse_float16_size_kb = save_binary_model(
    stripped_sparse_float16_tflite,
    "stripped_sparse_float16.tflite"
)
stripped_sparse_float16_acc, stripped_sparse_float16_preds = evaluate_tflite_model(
    stripped_sparse_float16_tflite,
    X_test,
    y_test
)

```

Why this is useful. This pattern allows students to stack multiple compression ideas rather than treating each technique in isolation.

4.5 Pruning a distilled student

In the knowledge distillation notebook, pruning is applied to the already-trained distilled student by cloning it first and then copying weights.

```

student_for_pruning = keras.models.clone_model(distilled_student)
student_for_pruning.set_weights(distilled_student.get_weights())

pruned_distilled_model = prune_low_magnitude(
    student_for_pruning,
    **pruning_params
)

```

Why cloning is a good practice. This preserves the original distilled student model so you can still compare:

- distilled model before pruning,
- pruned model with mask,
- stripped sparse model,
- sparse + quantized model.

5. Knowledge Distillation (Output-Based)

5.1 Teacher and student models

The knowledge distillation notebook uses a larger teacher and a smaller student.

```

def build_teacher_model(input_dim, num_classes):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(512, activation="relu"),
        layers.Dense(256, activation="relu"),
        layers.Dense(128, activation="relu"),
        layers.Dense(num_classes, activation="softmax"),
    ])
    return model

```

```
def build_student_model(input_dim, num_classes):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(128, activation="relu"),
        layers.Dense(64, activation="relu"),
        layers.Dense(num_classes, activation="softmax"),
    ])
    return model
```

Design intuition. The teacher is larger and more expressive. The student is smaller and cheaper to deploy. Distillation tries to transfer the teacher's knowledge into the smaller student.

5.2 Baseline student training before distillation

Before distillation, the smaller student is trained normally using the hard labels only.

```
student_baseline_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

student_baseline_history = student_baseline_model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=64,
    callbacks=student_callbacks,
    verbose=1
)
```

Why this baseline is necessary. Without this comparison point, you cannot tell whether knowledge distillation actually helped.

5.3 Custom distillation model wrapper

The core implementation idea is a custom `keras.Model` subclass that combines:

- **student loss** using the true labels,
- **distillation loss** comparing softened teacher and student outputs,
- a weighted sum controlled by **alpha** and **temperature**.

```
class Distiller(keras.Model):
    def __init__(self, student, teacher):
        super().__init__()
        self.teacher = teacher
        self.student = student

    def compile(
        self,
```

```

optimizer,
metrics,
student_loss_fn,
distillation_loss_fn,
alpha=0.3,
temperature=4.0,
):
    super().compile(optimizer=optimizer, metrics=metrics)
    self.student_loss_fn = student_loss_fn
    self.distillation_loss_fn = distillation_loss_fn
    self.alpha = alpha
    self.temperature = temperature

def train_step(self, data):
    x, y = data
    teacher_predictions = self.teacher(x, training=False)

    with tf.GradientTape() as tape:
        student_predictions = self.student(x, training=True)

        student_loss = self.student_loss_fn(y, student_predictions)

        distillation_loss = self.distillation_loss_fn(
            tf.nn.softmax(teacher_predictions / self.temperature, axis=1),
            tf.nn.softmax(student_predictions / self.temperature, axis=1),
        ) * (self.temperature ** 2)

        loss = self.alpha * student_loss + (1.0 - self.alpha) * distillation_loss

    trainable_vars = self.student.trainable_variables
    gradients = tape.gradient(loss, trainable_vars)
    self.optimizer.apply_gradients(zip(gradients, trainable_vars))

    self.compiled_metrics.update_state(y, student_predictions)

    results = {m.name: m.result() for m in self.metrics}
    results.update({
        "student_loss": student_loss,
        "distillation_loss": distillation_loss,
    })
    return results

def test_step(self, data):
    x, y = data
    y_prediction = self.student(x, training=False)

    student_loss = self.student_loss_fn(y, y_prediction)
    self.compiled_metrics.update_state(y, y_prediction)

    results = {m.name: m.result() for m in self.metrics}
    results.update({"student_loss": student_loss})
    return results

```

Meaning of the key settings.

- `student_loss_fn`: standard supervised loss on the true labels.
- `distillation_loss_fn`: soft-target loss, here implemented with KL divergence.
- `alpha`: balances hard-label learning and teacher imitation.
- `temperature`: softens the teacher and student probability distributions.

5.4 Compiling and training the distiller

```
distilled_student = build_student_model(num_features, num_classes)

distiller = Distiller(student=distilled_student, teacher=teacher_model)
distiller.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="accuracy")],
    student_loss_fn=keras.losses.SparseCategoricalCrossentropy(),
    distillation_loss_fn=keras.losses.KLDivergence(),
    alpha=0.3,
    temperature=4.0,
)

distillation_history = distiller.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=64,
    callbacks=distillation_callbacks,
    verbose=1
)
```

Practical interpretation. A smaller `alpha` puts more emphasis on teacher guidance. A larger `alpha` puts more emphasis on the ground-truth labels. The notebook uses a blended approach.

5.5 Final compression after distillation

The notebook finally combines multiple steps:

1. train the teacher,
2. train the baseline student,
3. distill the student,
4. prune the distilled student,
5. strip pruning wrappers,
6. quantize the stripped sparse student to int8.

```
converter = tf.lite.TFLiteConverter.from_keras_model(stripped_pruned_model)
converter.optimizations = [
    tf.lite.Optimize.DEFAULT,
```

```
tf.lite.Optimize.EXPERIMENTAL_SPARSITY
]
converter.representative_dataset = representative_data_gen
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8

stripped_sparse_int8_tflite = converter.convert()
```

Why this pipeline is powerful. This is a realistic TinyML workflow: first reduce model redundancy through distillation, then add sparsity, and finally convert to a deployment-friendly integer format.

Closing Notes

For most student projects, a good workflow is:

1. start with a clean baseline model and verify training and evaluation,
2. convert to FP32 TFLite and verify correctness,
3. try PTQ variants,
4. if needed, move to QAT for better int8 robustness,
5. apply pruning carefully and remember to strip the mask,
6. optionally combine pruning, quantization, and knowledge distillation.

This document is meant to be used as a reusable implementation template. Students should still adapt architecture size, input shape, representative data, and compression hyperparameters to match the specific dataset and deployment target of their project.